HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

PHÂN TÍCH THIẾT KẾ HỆ THỐNG

CHƯƠNG 6: THIẾT KẾ LỚP

ONE LOVE. ONE FUTURE.

Troy University Resources

Nội dung

- Thiết kế lớp

Nội dung

- Thiết kế lớp

Troy University Resources

Các hoạt động thiết kế đối tượng

- Mở rộng và tiếp tục phát triển các mô hình phân tích.
- Tạo các mô tả cho thiết kế chi tiết lớp và phương thức
  - Bổ xung các đặc tả cho mô hình hiện có
  - Xác định các tiềm năng tái sử dụng
  - Tái cấu trúc thiết kế
  - Tối ưu hóa thiết kế
  - Ánh xạ các lớp lĩnh vực ứng dụng và ngôn ngữ triển khai
- Thay đổi 1 lớp trên 1 tầng có thể khiến các lớp khác (có thể trên các tầng khác) có gắn kết với lớp đó thay đổi theo.

Các đặc tả bổ xung

- Kiểm tra các mô hình cấu trúc và hành vi để đảm bảo các thành phần là đầy đủ và cần thiết.
- Thiết lập giới hạn nhìn của các thuộc tính và phương thức của mỗi lớp (gắn với ngôn ngữ triển khai).
- Xác định nguyên mẫu của các phương thức: Tên, các tham số, kiểu trả về.
- Xác định các ràng buộc mà đối tượng phải đáp ứng
  - Có 3 loại ràng buộc: Tiền điều kiện, hậu điều kiện, tính chất bất biến.
  - *(Có thể được mô tả bằng OCL).*
- Các thông tin bổ xung được mô tả dưới dạng hợp đồng, hoặc được thêm vào thẻ CRC và biểu đồ lớp.

Troy University Resources

Các đặc tả bổ xung(2)

- Chúng ta cũng cần xác định cách xử lý các ngoại lệ:
  - *(phát sinh nếu các ràng buộc không được đảm bảo)*
  - Hệ thống có thể xử lý đơn giản bằng cách bỏ qua?
  - Hoàn tác các thay đổi đã thực hiện dẫn đến ngoại lệ?
  - Để người dùng lựa chọn phương án xử lý ngoại lệ nếu có nhiều phương án?
- Người thiết kế phải xác định các lỗi mà hệ thống phải xử lý
  - Gắn kết với ngôn ngữ lập trình
  - Sử dụng mã lỗi
  - Sử dụng cơ chế exception (như trong C++ hoặc Java)

Xác định các tiềm năng tái sử dụng

- Trong pha thiết kế, bên cạnh việc sử dụng các mẫu phân tích, chúng ta có thể sử dụng:
  - Các mẫu thiết kế, các nền tảng, các thư viện, các thành phần.
- Tiềm năng tái sử dụng có thể thay đổi theo từng tầng
  - Ví dụ: 1 thư viện có thể ít hữu ích trong tầng lĩnh vực ứng dụng, nhưng lại rất hữu ích trong tầng nền tảng.
- Các mẫu thiết kế
  - Rất hữu ích để gom nhóm các lớp tương tác có thể cung cấp giải pháp cho 1 vấn đề phổ biến. Có thể giải quyết "1 vấn đề thiết kế điển hình trong 1 ngữ cảnh cụ thể".
  - *(Các chi tiết sẽ được cung cấp sau).*

Troy University Resources

Tiềm năng tái sử dụng: Nền tảng và thư viện

- Nền tảng cung cấp 1 tập lớp có thể được sử dụng làm cơ sở để phát triển chương trình ứng dụng
  - Có thể triển khai toàn bộ hoặc 1 phần của hệ thống.
  - CORBA, DCOM, Spring Boot, Struts, Qt, v.v..
- Thư viện bao gồm 1 tập lớp, tương tự nền tảng, được thiết kế để tái sử dụng
  - Có nhiều thư viện hỗ trợ các mảng nghiệp vụ
    - Ví dụ: Xử lý số học và thống kê; Phát triển giao diện đồ họa (tầng HCI); Kết nối với CSDL (tầng quản lý dữ liệu - DAM).
- Nền tảng và thư viện thường cho phép kế thừa các lớp của nó, chúng ta có thể kế thừa để tái sử dụng lớp hoặc tạo đối tượng trực tiếp từ các lớp đã có.
  - *(Làm tăng tính ghép nối).*

Tiềm năng tái sử dụng: Thành phần

- Thành phần/component:
  - Mảnh phần mềm đầy đủ, được đóng gói để có thể nhúng vào 1 hệ thống để cung cấp 1 tập chức năng cụ thể.
    - Ví dụ các thành phần được triển khai dựa trên các công nghệ ActiveX hoặc JavaBean.
  - Cung cấp 1 tập phương thức giao diện để tương tác với các đối tượng có trong nó.
    - Lô-gic hoạt động bên trong thành phần được ẩn sau các API.
  - Có thể được triển khai bằng các lớp thư viện và nền tảng, nhưng cũng có thể được sử dụng để triển khai nền tảng.
  - Nếu giao diện không thay đổi, thì cập nhật phiên bản mới của thành phần thường không yêu cầu thay đổi mã nguồn
    - Có thể không yêu cầu biên dịch lại.
  - Thường được sử dụng để giản lược các chi tiết triển khai.

Troy University Resources

Tái cấu trúc thiết kế

- Đóng gói phần chung (factoring)
  - Lớp mới có thể được gắn kết với các lớp cũ thông qua kế thừa, tổng hợp hoặc liên kết
    - Lưu ý các vấn đề ghép nối (coupling), thống nhất (cohesion), và đồng sinh (connascence).
- Triển khai các mối quan hệ trên biểu đồ lớp như những thuộc tính.
  - Các ngôn ngữ hướng đối tượng hầu như không phân biệt các quan hệ tổng hợp hay liên kết.
    - Các mối quan hệ liên kết và tổng hợp phải được chuyển thành các thuộc tính trong lớp.

Đóng gói phần chung

- Tách và đóng gói phần giống nhau và khác nhau của các thứ được quan tâm
- Các lớp mới được hình thành thông qua:
  - Quan hệ khái quát hóa
    - Tạo lớp bậc cao hơn (ví dụ, tạo lớp Employee từ tập vị trí công việc)
  - Chi tiết hóa
    - Tạo lớp cụ thể (ví dụ, tạo lớp Secretary hoặc Bookkeeper từ lớp Employee)
  - Quan hệ tổng hợp (có các thành phần)

Troy University Resources

Tối ưu hóa thiết kế

Có thể được thực hiện để tạo thiết kế hiệu quả hơn

- Kiểm tra các đường dẫn tới đối tượng:
  - Có những trường hợp thông điệp từ 1 đối tượng tới 1 đối tượng khác phải qua nhiều bước trung gian.
  - Nếu đường dẫn dài và thông điệp được gửi thường xuyên, thì cần cân nhắc rút ngắn đường truyền thông điệp.
    - Có thể bổ xung thuộc tính vào đối tượng gửi thông điệp để có thể truy cập trực tiếp.

Tối ưu hóa thiết kế: Thuộc tính

- Xác định những phương thức sử dụng thuộc tính và những đối tượng sử dụng phương thức.
- Nếu chỉ có phương thức đọc và cập nhật sử dụng thuộc tính, và chỉ có đối tượng thuộc 1 lớp gửi thông điệp đọc và cập nhật đối tượng
  - Thuộc tính đó có thể thuộc về lớp gọi thay vì lớp được gọi
  - Chuyển thuộc tính sang lớp gọi có thể giúp hệ thống hoạt động nhanh hơn.

Troy University Resources

Tối ưu hóa thiết kế: Luồng ra

- Luồng ra (fan-out) bao gồm các thông điệp được gửi trực tiếp và gián tiếp trong thời gian thực hiện phương thức
  - Thông điệp trực tiếp được gửi bởi chính phương thức đó
  - Thông điệp gián tiếp được gửi bởi phương thức được gọi trong khi thực hiện phương thức đó
- Nếu kích thước luồng ra quá lớn so với các phương thức khác trong hệ thống, thì phương thức đó có thể cần được tối ưu hóa.

Tối ưu hóa thiết kế: Thứ tự thực hiện

- Kiểm tra thứ tự thực hiện các lệnh trong phương thức được sử dụng thường xuyên
  - Có những trường hợp có thể sắp xếp lại các câu lệnh để đạt được hiệu quả cao hơn.
  - Ví dụ: Giả định kịch bản tìm kiếm, trong đó phạm vi tìm kiếm được thu hẹp sau khi tìm kiếm theo 1 thuộc tính.
    - Có thể tìm cách tối ưu hóa xử lý theo 1 thứ tự thuộc tính tối ưu.

Troy University Resources

Tối ưu hóa thiết kế: Sử dụng bộ nhớ đệm

- Tránh tính toán lại thuộc tính suy diễn:
  - Ví dụ tạo thuộc tính tổng/total để lưu tổng giá trị đơn hàng.
  - Thiết lập cơ chế kích hoạt tiến trình tính toán.
  - Chỉ tính lại giá trị của thuộc tính suy diễn khi thay đổi các thuộc tính thành phần được sử dụng trong tính toán.

Ánh xạ các lớp lĩnh vực ứng dụng

- Tái cấu trúc các thành phần đa kế thừa nếu ngôn ngữ chỉ hỗ trợ đơn kế thừa
- Tái cấu trúc các thành phần kế thừa nếu ngôn ngữ không hỗ trợ kế thừa
- Tránh triển khai 1 thiết kế hướng đối tượng với 1 ngôn ngữ lập trình không hỗ trợ lập trình hướng đối tượng

Troy University Resources

Các ràng buộc và các hợp đồng

- Hợp đồng là 1 tập các ràng buộc (constraints) và các đảm bảo (guarantees)
  - Nếu đối tượng gửi (client) đáp ứng các ràng buộc, thì đối tượng cung cấp dịch vụ (server) đảm bảo hành vi mong đợi
- Hợp đồng mô tả thông điệp được gửi giữa các đối tượng
  - Được tạo cho các phương thức công khai của lớp
  - Cần chứa đủ thông tin để người lập trình có thể hiểu hành vi mong đợi của phương thức
- Các loại ràng buộc:
  - Tiền điều kiện - Phải đúng trước khi phương thức được thực hiện
  - Hậu điều kiện - Phải đúng sau khi phương thức kết thúc
  - Bất biến - Phải luôn đúng đối với tất cả các đối tượng của một lớp (vd tính chất của thuộc tính)

Biểu diễn tính chất bất biến (1)

Back:

Attributes:

Order Number	(1..1)	(unsigned long)	
Date	(1..1)	(Date)	
Sub Total	(0..1)	(double)	{Sub Total = ProductOrder. sum(GetExtension())}
Tax	(0..1)	(double)	(Tax = State.GetTaxRate() * Sub Total)
Shipping	(0..1)	(double)	
Total	(0..1)	(double)	
Customer	(1..1)	(Customer)	
Cust ID	(1..1)	(unsigned long)	{Cust ID = Customer. GetCustID()}
State	(1..1)	(State)	
StateName	(1..1)	(String)	{State Name = State. GetState()}

Relationships:

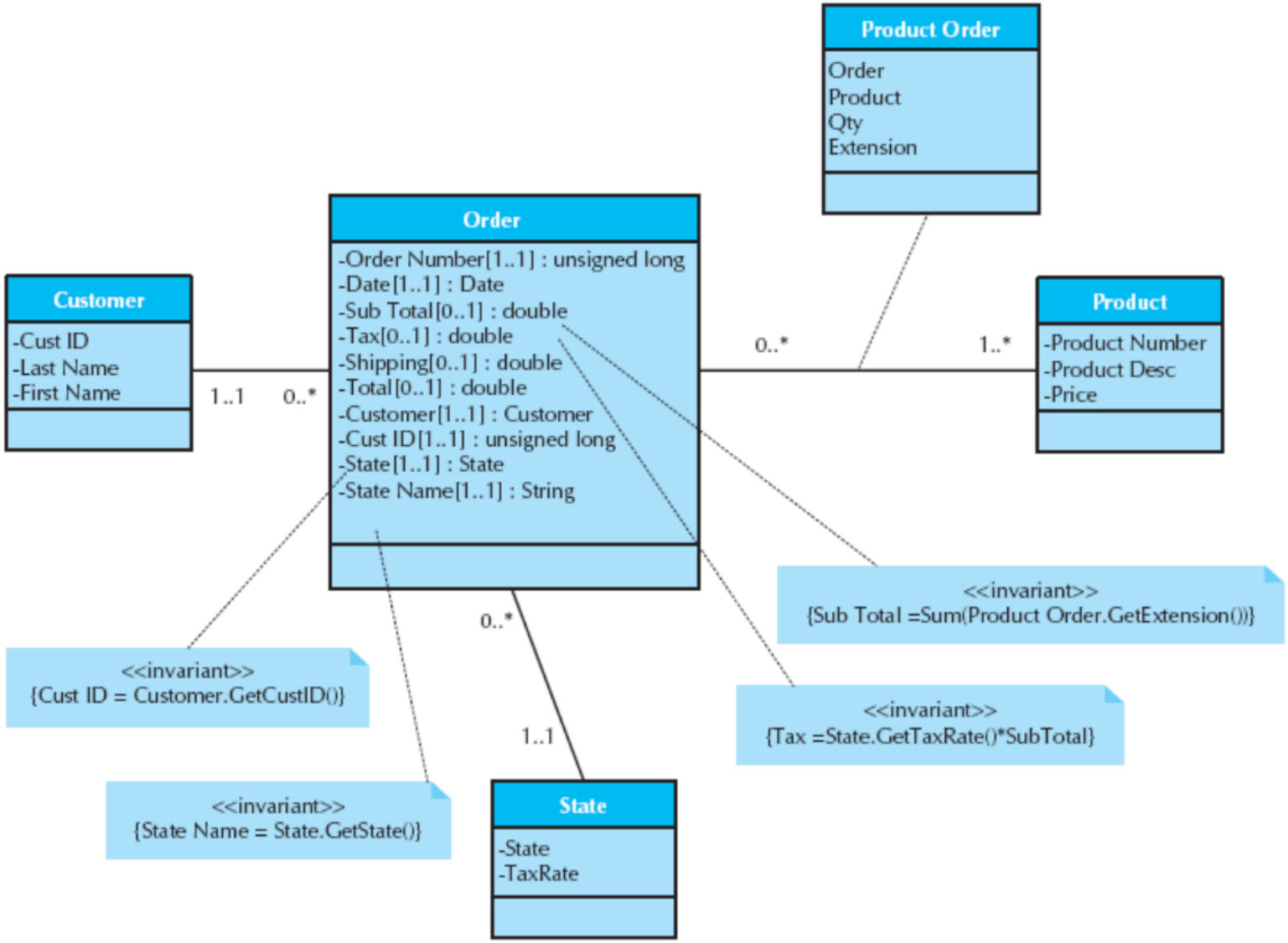
Generalization (a-kind-of):

Aggregation (has-parts):

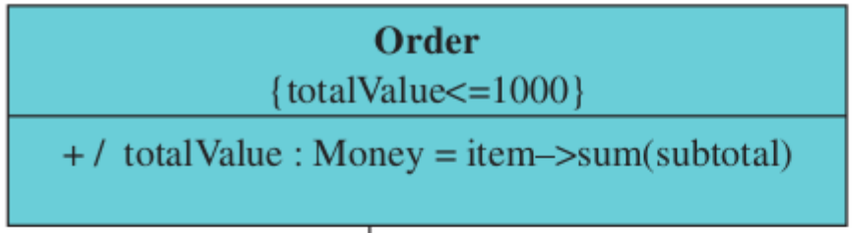
Other Associations: Customer {1..1} State {1..1}Product {1..*}

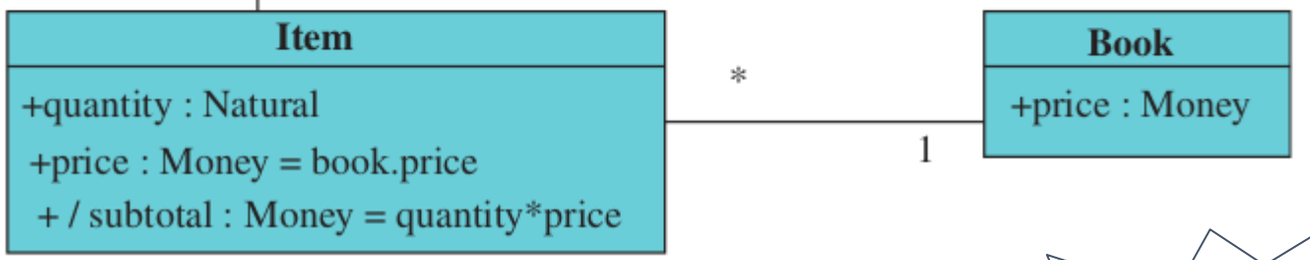
Troy University Resources

Biểu diễn tính chất bất biến(2)



Biểu diễn tính chất bất biến (3)






```
Context Order::totalValue:Money
  derive:
    self.item->sum(anItem|anItem.subtotal)

Context Item::subtotal:Money
  derive:
    self.quantity*self.price
```

Các mô tả bằng OCL

```
Context Item::price:Money
  init:
    self.book.price
```


Troy University Resources

Biểu mẫu hợp đồng thông điệp

(Phi chuẩn, đặc tả thông điệp)

Tên phương thức:	Tên lớp:	Mã số:
Mã khách:		
Ca sử dụng liên quan:		
Mô tả các trách nhiệm:		
Các tham số nhận được:		
Kiểu dữ liệu trả về:		
Tiền điều kiện:		
Hậu điều kiện:		

Đặc tả phương thức

- Mô tả các chi tiết của mỗi phương thức
  - Để người lập trình viết mã nguồn
  - *(đây là tài liệu thiết kế rất gần triển khai)*
- Không có quy chuẩn, tuy nhiên có thể bao gồm các thông tin:
  - Thông tin tổng quan (Tên phương thức, tên lớp, v.v...)
  - Sự kiện - Kích hoạt phương thức (ví dụ khi nhấn chuột)
  - Cấu trúc thông điệp được truyền tới bao gồm cả các tham số và giá trị trả về
  - Đặc tả giải thuật
  - Các thông tin liên quan khác

Troy University Resources

Biểu mẫu đặc tả phương thức

Tên phương thức:	Tên lớp:	ID:
Mã thỏa thuận:	Lập trình viên:	Thời hạn:
Ngôn ngữ lập trình: ☐ Visual Basic ☐ Smalltalk ☐ C++ ☐ Java		
Kích hoạt/Sự kiện:		

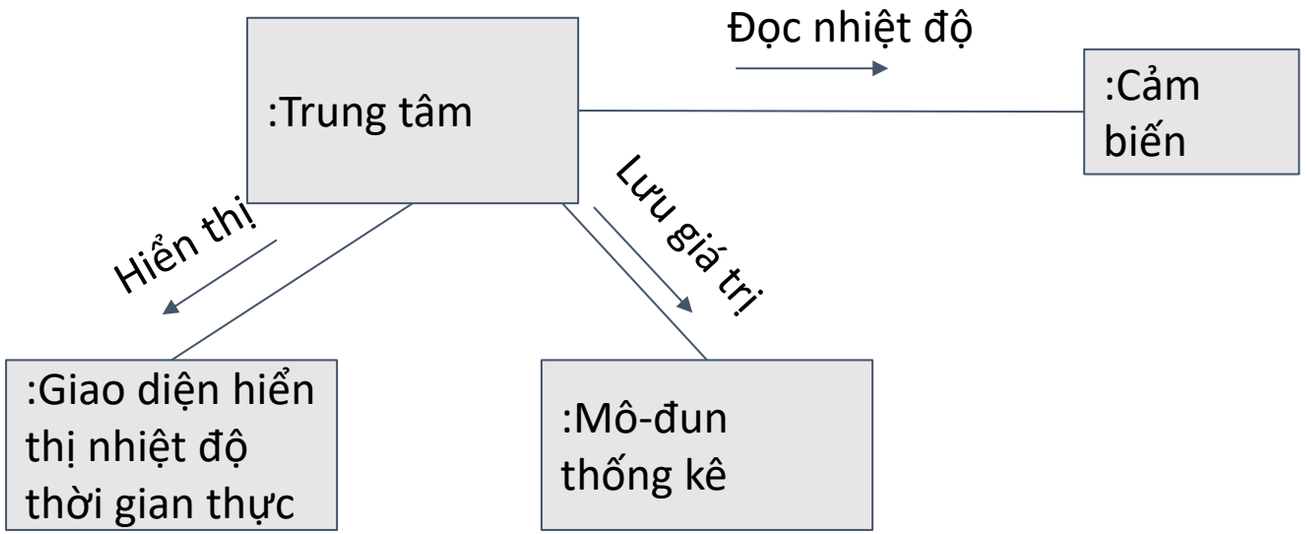
Các tham số nhận được: Kiểu dữ liệu:	Ghi chú:

Các thông điệp đã gửi & Các tham số đã truyền: TênLớp.TênPhươngThức:	Kiểu dữ liệu của tham số:	Ghi chú:

Tham số trả về: Kiểu dữ liệu:	Ghi chú:
Đặc tả giải thuật:	
Các ghi chú khác:	

Bài tập

Phân tích các vấn đề với thiết kế sau và phác thảo giải pháp hướng tới liên kết lỏng giữa các thành phần: Vẽ sơ đồ lớp và sơ đồ tuần tự.



Troy University Resources

HUST

hust.edu.vn fb.com/dhbkhn

THANK YOU !